

Motion Detection System

System Architecture and End-to-End Design Guide

Project goal: Build a local Python and OpenCV desktop app that detects general visible movement in a live webcam feed and displays a message on screen. It does not identify people, objects, or movement types.

Scope assumption: “All kinds of movements” means sufficiently visible changes between video frames. Detection sensitivity depends on camera stability, lighting, and chosen thresholds. The app processes frames locally and does not save or transmit video.

1. Requirements and scope

Functional requirements

- Open the webcam and display a live preview.
- Detect changed regions between consecutive frames using a configurable threshold and minimum region size.
- Show “Motion detected!” when movement is present and “No motion detected” when the scene is quiet; optionally outline detected regions.
- Allow the user to quit with a key or the window close control.
- Handle camera startup/read failures with a clear message and clean shutdown.

Quality requirements

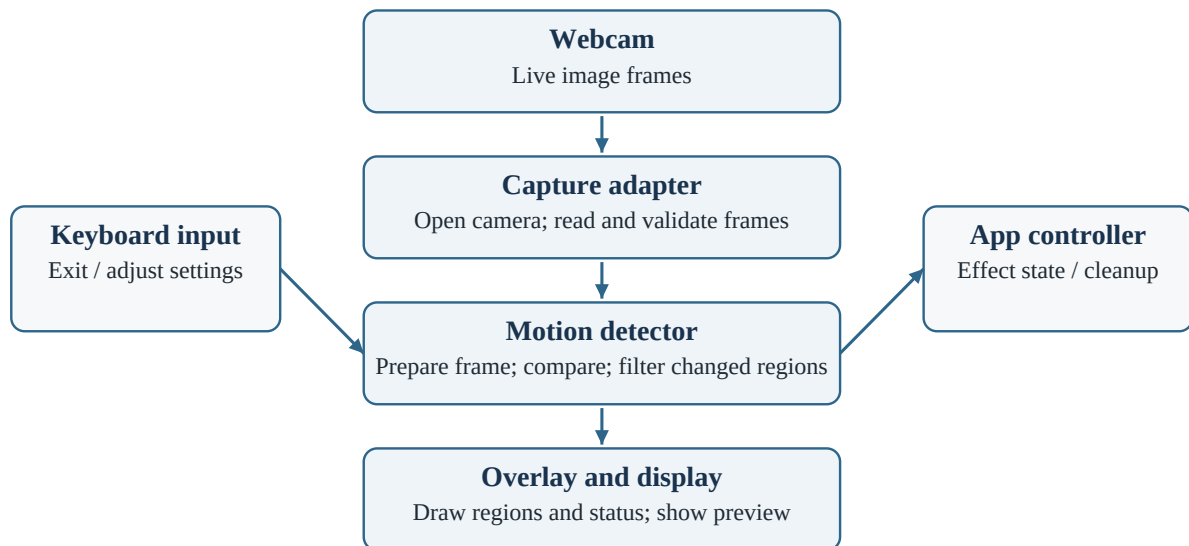
- Reliability: reject invalid frames and release camera resources on all normal and handled failure paths.
- Responsiveness: avoid unnecessary processing or work in the frame loop; measure on target devices instead of promising a frame rate.
- Usability: make current status and exit controls easy to understand.
- Privacy: process images in memory only; do not record, upload, or include frames in logs.
- Maintainability: keep camera access, motion analysis, display, and app lifecycle in distinct responsibilities.

Version 1 boundaries

One camera, one preview window, general movement detection, a status message, and optional region boxes. No object recognition, face analysis, recording, accounts, network service, or database.

2. System architecture

The app runs as one local process. The detector compares the current prepared frame with the previous prepared frame, creates a binary motion mask, filters small changes, and reports whether meaningful changed regions remain. The display shows the camera image with a status overlay.



*On exit or handled failure, release the camera and close the display window.
Figure 1. Local webcam motion detection architecture.*

Component responsibilities

Component	Responsibility
Application controller	Coordinates startup, frame loop, detector state, keyboard actions, user messages, errors, and cleanup.
Camera adapter	Opens the configured/default camera, reads frames, reports camera status, and releases the device.
Frame preprocessor	Converts frames to grayscale and applies light blur to reduce minor sensor noise before comparison.
Motion detector	Compares consecutive frames, thresholds changed pixels, removes small noise, and returns detected regions and a motion flag.
Overlay and display	Draws optional region boxes and status text, displays the preview, and reads keyboard/window events.
Configuration	Provides validated settings such as camera index, pixel-change threshold, minimum region area, and quiet/reset timing.

3. Motion detection logic

A simple first version can use consecutive-frame differencing. It is easy to explain and works for general movement, but camera shake and abrupt lighting changes can also appear as motion. A later version could use a background model if the project needs to detect movement against a more stable scene.

Frame-by-frame algorithm

Step	Processing
1. Read	Get the next camera frame; if the read fails, stop processing and report the issue.
2. Prepare	Convert the frame to grayscale and blur slightly to reduce pixel-level noise.
3. Initialize or compare	For the first valid frame, store it as the previous frame. Otherwise calculate the absolute difference from the previous prepared frame.
4. Create mask	Apply a pixel-change threshold to create a binary mask of changed areas.
5. Filter regions	Use morphological cleanup to reduce isolated noise; find contours and ignore regions smaller than a configurable area.
6. Decide and update	If any regions remain, mark motion present. Store the current prepared frame for the next comparison.
7. Display	Draw optional boxes and show the status message; keep “motion detected” visible briefly after the last detected frame to reduce flicker.

Important behavior

- The first frame establishes a comparison baseline and should not trigger an alert by itself.
- Frame differencing detects change between adjacent frames. An object that stops moving will no longer trigger movement after the scene stabilizes.
- Threshold and minimum-area settings trade sensitivity against false alerts; tune them using the target camera and environment.
- A camera bump or sudden lighting change can produce many changed pixels. Consider a short settling period or a future background-model approach if these false alerts matter.

4. Runtime flow and failure handling

Startup: initialize configuration and display, open the camera, verify it is ready, and initialize previous-frame state. If opening fails, show a clear message and exit.

Loop: read and validate frame → preprocess → detect and filter motion → update alert state → draw message/regions → display → poll controls → repeat.

Shutdown: on quit, window close, or handled failure, release the camera and destroy OpenCV windows in a guaranteed cleanup path.

Condition	Expected behavior
Camera unavailable	Show a short explanation and stop cleanly.
Frame read fails	Do not process a missing/invalid frame; report the failure and clean up.
No significant changes	Show “No motion detected.”
Movement detected	Show “Motion detected!” and optional boxes around qualifying regions.
Quit key / window close	Leave the loop and release camera and window resources.

5. User interface and accessibility

- Keep the live preview central and show status text in a consistent, high-contrast position.
- Do not rely on color alone; include readable status text.
- Document the quit key and any adjustable sensitivity controls.
- Do not let the alert obscure the main area of the video. If the user closes the window, run the same cleanup path as a quit key.

6. Privacy and security

Webcam images may contain sensitive information. The first release should keep frames in memory and discard them after processing. Do not add recording, screenshots, uploads, analytics, or frame-containing crash reports without a separate product and privacy review. Logs should contain operational details only, such as camera open failures and app shutdown.

7. Code organization and development

```
webcam-motion-app/  
■■■■ README.md  
■■■■ pyproject.toml (or pinned requirements)  
■■■■ src/motion_app/  
■ ■■■■ main.py  
■ ■■■■ app.py  
■ ■■■■ camera.py  
■ ■■■■ preprocessing.py  
■ ■■■■ motion_detector.py  
■ ■■■■ display.py  
■ ■■■■ config.py  
■■■■ tests/  
■■■■ test_motion_detector.py  
■■■■ test_app_flow.py
```

- Start with a working webcam preview before implementing detection.
- Build the detector as a function or class that accepts image frames and returns a motion flag plus regions; it should not open a camera or UI window.
- Keep configuration validated and document defaults. Pin tested dependency versions.
- Use version control and small changes; review code, test results, and AI-generated suggestions before accepting them.

8. Testing and acceptance

Tests

- Unit tests with synthetic frames: identical frames should produce no regions; a sufficiently large changed block should be detected; tiny isolated changes should be filtered according to the chosen minimum area.
- Component tests with a fake frame source: cover successful reads, failed reads, status updates, and cleanup without a physical camera.
- Manual checks on each target platform: camera permissions, steady scene, movement near and far from the camera, lighting changes, camera bump, window close, quit key, and camera unavailable.
- Measure responsiveness on actual target machines and record test conditions; do not claim an unmeasured frame rate.

Version 1 acceptance checklist

- Live webcam preview works when a camera is available.
- Visible movement produces an on-screen alert and optional region boxes.
- The alert returns to the quiet state after movement stops, without excessive flicker.
- Small image noise is filtered; limitations under lighting changes and camera movement are documented.
- The app handles camera errors and releases resources on exit.
- Video is not stored or transmitted; setup and run instructions let another student use the app.

9. Release and ownership

For this desktop project, release means another person can install or run the app reliably. Provide setup instructions, tested dependency versions, control instructions, supported platform notes, known limitations, and a version identifier. If packaging as an executable is added, test camera permissions and OpenCV display behavior on each intended platform.

- Maintain the repository, review dependency updates, and retest camera behavior after changes.
- Ask for issue details such as app version, operating system, camera model if known, reproduction steps, and error text. Do not request webcam footage by default.
- Use synthetic frames or non-sensitive diagnostic information to reproduce issues where possible.
- Before expanding scope, decide whether camera selection, threshold controls, a background model, or packaging is needed and document the reason.

Student handoff deliverables

- Product scope and requirements.
- Reviewed architecture and key design decisions.
- Readable source code and reproducible setup instructions.
- Automated tests and a manual test checklist.
- Release/demo instructions, known limitations, privacy behavior, and an owner for maintenance.